AI / ML • NETWORK SECURITY PROJECT • PROJECT REPORT

ML-Based Network Intrusion Detection

A controlled benchmark of five classifiers on corrected CICIDS2017, with explainability, deployability and generalisation tests

Ahmad Azim & Pawan Kumar
Computer Networks — Lab Project
October 2026

Complete record of the project: proposal, design, implementation, experiments, results and task division.

Contents

1. Executive Summary 4

2. Project Overview 5

2.1 Problem . . . 5

2.2 Research questions . . . 5

2.3 Hypotheses (stated in the proposal before any experiment) . . . 5

2.4 What changed from the proposal, and why . . . 5

3. Team and Task Division 6

3.1 Ahmad Azim — Data engineering, pipeline infrastructure and generalisation . . . 6

3.2 Pawan Kumar — Modelling, evaluation, explainability and reporting . . . 6

3.3 Shared work . . . 7

3.4 Who presents what . . . 7

4. How We Did It: Project Timeline 8

5. Tools and Environment 9

5.1 Repository layout . . . 9

5.2 Design principles . . . 9

6. Datasets 10

6.1 Corrected CICIDS2017 (training and testing) . . . 10

6.2 Corrected CSE-CIC-IDS2018 (cross-dataset test only) . . . 11

6.3 Datasets considered and rejected . . . 11

7. Methodology 12

7.1 Label mapping . . . 12

7.2 Cleaning . . . 12

7.3 Train/test splits . . . 13

7.4 Feature selection (training split only) . . . 13

7.5 Classifiers and hyper-parameters . . . 14

7.6 Class-imbalance strategies . . . 14

7.7 Training protocol . . . 15

7.8 Evaluation metrics . . . 15

7.9 Significance tests . . . 15

7.10 Explainability, deployability and generalisation tests . . . 15

8. Results 16

8.1 Random split — all runs . . . 16

8.2 Time-block split — all runs . . . 16

8.3 Per-class recall . . . 17

8.4 Effect of the imbalance strategies . . . 18
8.5 Statistical significance . . . 18
8.6 Deployability . . . 20
8.7 Explainability (SHAP) . . . 20
8.8 Leave-one-attack-out (unseen attack families) . . . 21
8.9 Cross-dataset generalisation (CSE-CIC-IDS2018, no retraining) . . . 22
9. Hypotheses: Verdicts **24**
Answers to the research questions . . . 24
10. Discussion and Limitations **24**
11. Problems Encountered and How We Solved Them **24**
12. Conclusions and Future Work **26**
Recommendations . . . 26
Future work . . . 26
Appendix A. Reproducing the Results **27**
Pipeline steps . . . 27
Appendix B. Configuration (configs/default.yaml) **27**
Appendix C. References **29**

1. Executive Summary

Signature-based intrusion detection systems (Snort, Suricata) only catch attacks that already have a rule. This project built and rigorously evaluated a machine-learning network intrusion detection system (ML-NIDS) that classifies network **flows** as benign or as one of several attack families, using the statistical features produced by CICFlowMeter. Five classifiers — Logistic Regression, Random Forest, XGBoost, an RBF-kernel SVM and a multi-layer perceptron (MLP) — were trained under one identical, leakage-safe pipeline on the **corrected** CICIDS2017 dataset, each with three class-imbalance strategies (none, class weights, SMOTE).

Beyond accuracy, the study measures statistical significance, per-flow latency and model size, explains decisions with SHAP, and tests generalisation three ways: a time-separated split, a leave-one-attack-out experiment (a proxy for zero-day attacks), and an independent dataset (corrected CSE-CIC-IDS2018) evaluated with no retraining.

Measure	Value
Flows loaded / after cleaning	2,099,976 / 1,714,826
Experiment runs	14 per split (5 models × 3 strategies, MLP has no class weights) × 2 splits
Best macro-F1, random split	0.999 (Random Forest, SMOTE)
Best macro-F1, time-block split	0.938 (Random Forest, SMOTE)
Random Forest vs XGBoost	statistically tied (corrected t-test p = 0.92 random, 0.96 time-block)
Single-flow latency	XGBoost 0.30 ms, Random Forest 2.78 ms; ONNX ≈ 0.01–0.02 ms
Unseen attack families caught (default threshold)	≈ 0–2% for every family except DDoS
2018 attacks caught by RF / XGBoost	0.3% (in-distribution binary F1 ≈ 0.9998)

Headline findings

- **In-distribution, the task is nearly solved.** All non-linear models exceed 0.99 macro-F1 on the random split; Random Forest and XGBoost are statistically indistinguishable.
- **XGBoost is the best accuracy-to-cost choice:** Random-Forest-level accuracy at 9× lower single-flow latency and about 3× smaller model size.
- **Class-imbalance handling did not matter much** on the corrected, de-duplicated data: SMOTE gave no significant gain, and class weights hurt the linear models by flooding benign traffic with alerts.
- **Models miss what they have not seen.** Held-out attack families are almost never flagged, and a SQL-injection variant that the time-block split kept out of training was missed every time.
- **Benchmark accuracy does not transfer.** On CSE-CIC-IDS2018, Random Forest and XGBoost detect only 0.3% of attacks; the simplest model (Logistic Regression) transfers best. SHAP shows a testbed-specific feature (Bwd Init Win Bytes) near the top of the ranking, a likely cause.

2. Project Overview

2.1 Problem

Given network traffic represented as bidirectional flows, decide whether each flow is benign or malicious and, if malicious, which attack family it belongs to. The detector must combine high detection with a very low false-positive rate: at millions of flows per day even 1% false positives produces thousands of useless alerts (alert fatigue).

2.2 Research questions

- **RQ1** — Which of the five classifiers gives the best accuracy versus false-positive trade-off under an identical, reproducible pipeline?
- **RQ2** — How does that trade off against deployability: training time, per-flow latency and model size?
- **RQ3** — Which flow features drive the decisions, and are they defensible from a networking standpoint?
- **RQ4** — Does performance hold on data from a different capture (another dataset), or does it collapse?

2.3 Hypotheses (stated in the proposal before any experiment)

#	Hypothesis
H1	Tree ensembles (RF, XGBoost) beat Logistic Regression and match or exceed SVM / MLP on F1.
H2	Aggregate accuracy is > 98%, but rare-class recall (Infiltration, Heartbleed) is much lower before SMOTE.
H3	RF or XGBoost gives the best accuracy-to-latency trade-off; SVM has the highest training cost.
H4	Accuracy drops on an independent dataset, quantifying a generalisation gap.

2.4 What changed from the proposal, and why

Before writing any code, the proposal (August 2026) was reviewed. Six issues would have weakened the results; each was resolved as shown below. These changes are the main methodological contributions of the project.

Proposal said	Problem	What we did
Original CICIDS2017 CSVs	Documented labelling and flow-construction errors (Engelen et al. 2021; Liu et al. 2022)	Used the **corrected** CICIDS2017 re-extracted with a fixed CICFlowMeter
Cross-dataset on NSL-KDD or CICIDS2018	NSL-KDD has 41 features from a different extractor; cannot be aligned	Used **corrected CSE-CIC-IDS2018** (same fixed extractor, identical features)
SMOTE, scaling, RFE before CV	Fitting on data later used for validation leaks information	SMOTE inside each CV fold; feature selection on the training split only
Random split only	Near-identical neighbouring flows land in both train and test	Added a **time-block** split; duplicates removed before splitting; IPs/ports dropped
Per-class metrics for Heartbleed / Infiltration	11 and 36 flows: 2–7 test flows, i.e. noise	Excluded from multi-class (reported); kept in binary and LOAO tests
RBF-SVM on all data; paired t-test on folds	SVM cost grows ~n^2; plain t-test overstates significance	SVM capped at 60k rows; Nadeau–Bengio corrected t-test + McNemar

Two experiments were also added: **leave-one-attack-out** (to test the abstract's 'detects unseen attacks' claim directly) and **detection rate at a fixed false-positive rate**. The Raspberry Pi stretch goal was replaced by ONNX Runtime latency measurements, and the optional Scapy live-capture demo was dropped because Scapy cannot produce CICFlowMeter features.

3. Team and Task Division

The work is split into two balanced workstreams with a clean hand-off: **Ahmad Azim** owns everything up to a clean, split, feature-selected dataset plus the generalisation experiments that consume new data; **Pawan Kumar** owns the models, their evaluation and everything that explains or presents the results. The interface between them is the cached, prepared dataset (data/processed/) and the saved models (results/…/models/).

3.1 Ahmad Azim — Data engineering, pipeline infrastructure and generalisation

#	Task	How it was done	Code / output
A1	Proposal review & research design	Found the 6 issues in §2.4; chose datasets, splits and tests	README.md (decisions table)
A2	Environment & repository	Git repo, Python 3.12 venv, dependency list, libomp for XGBoost, ids command-line tool, YAML config	pyproject.toml, configs/, src/ids/cli.py, config.py
A3	Dataset acquisition	Downloaded corrected CICIDS2017 (328 MB); wrote a streamer that pulls single days out of the 9.7 GB 2018 archive with HTTP range requests, with retries	scripts/fetch_data.py
A4	Loading & label mapping	Uniform snake_case schema, float32, attack families, 'Attempted' → benign, Infiltration-Portscan → PortScan	src/ids/datasets.py
A5	Cleaning	NaN/inf removal, conflicting-label and exact de-duplication via row hashing, benign cap, rare-class handling	src/ids/preprocess.py
A6	Train/test splitting	Stratified random split; time-block split with StratifiedGroupKFold; fixed the bug that left DDoS out of the test set	src/ids/split.py
A7	Feature selection	Near-constant removal, |r| > 0.95 correlation pruning, RFE with Random Forest (train only)	src/ids/features.py
A8	Generalisation experiments	Leave-one-attack-out and cross-dataset evaluation on CSE-CIC-IDS2018	src/ids/experiments.py (run_loao, run_cross_dataset)
A9	Testing & automation	Synthetic-data generator, 22 end-to-end tests, overnight run script	src/ids/synthetic.py, tests/, scripts/run_full.sh

3.2 Pawan Kumar — Modelling, evaluation, explainability and reporting

#	Task	How it was done	Code / output
B1	Model implementation	Five classifiers in imblearn pipelines; log + standardise for LR/SVM/MLP; raw features for trees	src/ids/models.py
B2	Imbalance strategies	Class weights / balanced sample weights; SafeSMOTE (log-space neighbours, minorities → 20k rows)	src/ids/sampling.py
B3	Hyper-parameter tuning & training	RandomizedSearchCV (12 draws × 3 folds on 150k rows); quick run, then full runs on both splits	src/ids/experiments.py (tune, benchmark)
B4	Evaluation metrics	Macro/weighted F1, per-class tables, benign FPR, ROC/PR-AUC, recall at fixed FPR	src/ids/evaluate.py
B5	Statistical testing	Nadeau–Bengio corrected resampled t-test on 5-fold CV; McNemar on the test set	src/ids/stats.py
B6	Explainability	SHAP TreeExplainer: global ranking, beeswarm, per-flow waterfalls	src/ids/explain.py
B7	Deployability	Model size, single-thread single-flow latency, batch throughput, ONNX Runtime export	src/ids/deploy.py

#	Task	How it was done	Code / output
B8	Figures & notebooks	Confusion matrices, ROC/PR, per-class heatmaps, latency plots; 8 phase notebooks	src/ids/plots.py, notebooks/
B9	Reporting	Assessment deck (13 slides) and this report	docs/, scripts/build_*.py

3.3 Shared work

- Writing the proposal and the initial presentation (August 2026).
- Reviewing results together, deciding the hypothesis verdicts and recommendations (§9, §12).
- Demo recording and the final presentation.

3.4 Who presents what

Part of the presentation / viva	Presenter
Problem, motivation, dataset choice, preprocessing, leakage controls	Ahmad Azim
Split design, time-block split and its bug fix, feature selection	Ahmad Azim
Models, tuning, imbalance strategies, overall and per-class results, significance	Pawan Kumar
Deployability and SHAP explainability	Pawan Kumar
Leave-one-attack-out and cross-dataset generalisation	Ahmad Azim
Hypothesis verdicts, conclusions and recommendation	Both

4. How We Did It: Project Timeline

Chronological record from initialisation to results. Times are local (IST); durations are the measured wall-clock times on the project laptop.

When	Phase	What happened	Outcome
25 Aug 2026	Proposal	Proposal and 5-slide presentation submitted	docs/IDS_Project_Proposal.docx, .pptx
19 Oct	Review	Proposal and slides reviewed before building; 6 must-fix issues and several improvements identified (§2.4)	Agreed design
19 Oct	Dataset verification	Checked the NetFlow-v2 datasets: no CICIDS2017 version exists, so they cannot test a CICIDS2017-trained model. Switched target to corrected CSE-CIC-IDS2018; listed archive contents remotely (2017: 328 MB; 2018: 9.7 GB)	Cross-dataset plan fixed
19 Oct	Setup	Git repository, Python 3.12 virtual environment, dependencies; installed libomp so XGBoost loads on macOS	Working environment
19 Oct	Implementation	Wrote the src/ids package (13 modules), CLI, configs, synthetic data generator, 8 notebooks; 21 tests passing; fixed an inflated Random-Forest latency (thread-pool start-up per call)	Pipeline runs end-to-end
19 Oct	Real data	Downloaded corrected CICIDS2017; verified 91-column schema and all 28 label strings; remapped Infiltration-Portscan → PortScan; the prepare step ran in 95 s using 2 GB RAM	1.71M clean flows, 30 features
19–20 Oct	Quick benchmark	Small tuning budget, no CV: all 14 runs in 32 min	First results, timings
20 Oct 01:09	Full run	Random split 43 min, time-block split 62 min (cross-dataset step skipped: no 2018 data yet)	All models, stats, SHAP, LOAO
20 Oct	Bug fix	Time-block test set had 0 DDoS flows, and macro-F1 counted DDoS as 0. Splitter now keeps every class on both sides; macro metrics average over classes present; time-block run repeated	Valid time-block results
20 Oct	2018 data	First stream failed when the laptop slept (all 5 connections dropped at once). Added retries + caffeinate; 5 days streamed in parallel in 37 min	4,329,350 2018 flows
20 Oct	Cross-dataset	Every saved model evaluated on 2018 data, both splits (~20 min each)	Generalisation results
20 Oct	Assessment deck	13-slide deck: abstract, methodology, simulation results	docs/IDS_Assessment_Presentation.pptx
Oct 2026	Report	This document	docs/IDS_Project_Report.pdf

5. Tools and Environment

Component	Version / detail
Hardware	Apple MacBook, M1 (8 cores), 8 GB RAM, macOS
Language	Python 3.12 (virtual environment .venv/)
scikit-learn	1.6.1
xgboost	3.4.1
imbalanced-learn	0.14.2
shap	0.54.1
pandas	3.0.6
numpy	2.2.6
scipy	1.18.1
matplotlib	3.11.2
onnxruntime	1.31.0
skl2onnx	1.20.0
pyarrow	26.0.0
Version control	git (3 commits at the time of writing)

5.1 Repository layout

```
configs/        default.yaml (real runs), quick.yaml, smoke.yaml (synthetic)
src/ids/        datasets, preprocess, split, features, sampling, models, evaluate,
                stats, deploy, explain, plots, experiments, cli, synthetic
scripts/        fetch_data.py, run_full.sh, build_assessment_deck.py, build_report.py
notebooks/      01_eda ... 08_generalization
tests/          end-to-end tests on synthetic data (22 tests)
data/raw/       downloaded datasets (not in git)
data/processed/ cached cleaned/split/feature-selected data (not in git)
results/        metrics, predictions, models, figures (not in git)
docs/           proposal, slides, assessment deck, this report
```

5.2 Design principles

- **One config file** (configs/default.yaml) holds every setting, so every model sees the same data and the run is reproducible (seed 42).
- **Each step caches its output** to disk (prepare → benchmark → stats / deploy / explain / cross / loao → report), so steps can be re-run independently.
- **Nothing fitted on test data:** scaling, SMOTE and feature selection are fitted on training data only.
- **Tested before use:** a synthetic dataset with the real schema and quirks lets the whole pipeline run in about 2 minutes in tests.

6. Datasets

6.1 Corrected CICIDS2017 (training and testing)

CICIDS2017 (Canadian Institute for Cybersecurity) captures five days of realistic background traffic with staged attacks. The original CSVs contain documented errors in flow construction and labelling. We used the corrected release (Liu, Engelen et al., IEEE CNS 2022): the raw PCAPs re-processed with a fixed CICFlowMeter and relabelled. Each CSV has 91 columns: identifiers (flow ID, IPs, ports, timestamp), about 85 flow statistics, the label and an 'Attempted Category'. Flows that belong to an attack session but carry no attack payload are labelled '… - Attempted'; as the dataset authors recommend, they are treated as benign.

Day	Label	Flows
Monday	BENIGN	371,624
Tuesday	BENIGN	315,106
	FTP-Patator	3,972
	SSH-Patator	2,961
	SSH-Patator - Attempted	27
	FTP-Patator - Attempted	12
Wednesday	BENIGN	319,120
	DoS Hulk	158,468
	DoS GoldenEye	7,567
	DoS Slowloris	3,859
	DoS Slowhttptest - Attempted	3,368
	DoS Slowloris - Attempted	1,847
	DoS Slowhttptest	1,740
	DoS Hulk - Attempted	581
	DoS GoldenEye - Attempted	80
	Heartbleed	11
Thursday	BENIGN	288,172
	Infiltration - Portscan	71,767
	Web Attack - Brute Force - Attempted	1,292
	Web Attack - XSS - Attempted	655
	Web Attack - Brute Force	73
	Infiltration - Attempted	45
	Infiltration	36
	Web Attack - XSS	18
	Web Attack - SQL Injection	13
	Web Attack - SQL Injection - Attempted	5
Friday	BENIGN	288,544
	Portscan	159,066
	DDoS	95,144
	Botnet - Attempted	4,067

Day	Label	Flows
	Botnet	736

Total: 2,099,976 flows. Download: 328 MB zip, about 1.15 GB of CSV.

6.2 Corrected CSE-CIC-IDS2018 (cross-dataset test only)

Same research group and same fixed CICFlowMeter, so the feature columns match exactly; a different network, different hosts and different attack tools. The full archive is 9.7 GB (3–4 GB of CSV per day), more than the laptop's free disk, so scripts/fetch_data.py reads individual day files out of the remote zip with HTTP range requests, streams them in chunks, keeps every attack flow and a random 5% of benign flows, and writes a small parquet file per day.

Day	Attack flows kept	Benign flows kept (5%)	Families
friday-02-03-2018	142,921	307,850	Botnet
friday-16-02-2018	1,803,160	279,344	DoS
thursday-15-02-2018	31,050	268,887	DoS
wednesday-14-02-2018	94,197	290,005	BruteForce
wednesday-21-02-2018	1,084,023	293,694	DDoS

After cleaning: 4,329,350 flows (DoS 1,834,209, Benign 1,174,000, DDoS 1,084,023, Botnet 142,921, BruteForce 94,197). Because benign traffic was sampled at 5%, precision on this set is optimistic; recall and false-positive rate are unaffected.

6.3 Datasets considered and rejected

- **NSL-KDD** (proposal option): 41 features from a different 1999-era extractor; no meaningful mapping to CICFlowMeter features.
- **UQ NetFlow-v2 datasets**: share one feature set with each other, but contain no version of CICIDS2017, so a CICIDS2017 model cannot be tested on them.
- **Original CICIDS2017 / CSE-CIC-IDS2018 CSVs**: known labelling errors and slightly different column names.

7. Methodology

The pipeline has six stages. Every classifier receives exactly the same rows, split, features and metrics.

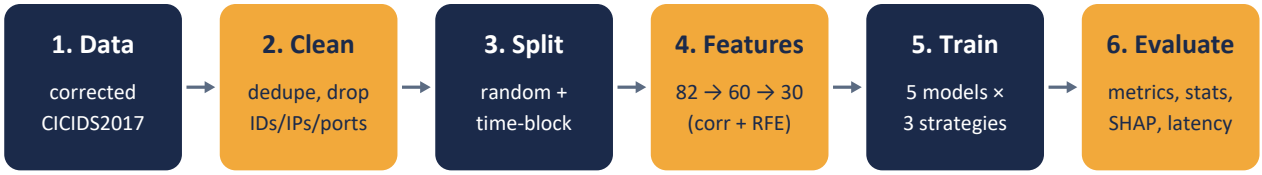


Figure 1. End-to-end pipeline.

7.1 Label mapping

Raw labels are mapped to attack families by ordered pattern rules, so the same code works for both datasets:

Raw labels	Family
BENIGN; any '… - Attempted' label	Benign
DoS Hulk, DoS GoldenEye, DoS Slowloris, DoS Slowhttptest	DoS
DDoS	DDoS
Portscan; **Infiltration - Portscan**	PortScan
FTP-Patator, SSH-Patator	BruteForce
Web Attack - Brute Force / XSS / SQL Injection	WebAttack
Botnet	Botnet
Infiltration	Infiltration
Heartbleed	Heartbleed

Why Infiltration-Portscan → PortScan: these 71,767 flows are the port scan launched from the already-compromised host. To a flow classifier they are port-scan traffic. Labelling them Infiltration would let 72k scan flows stand in for the 36 genuine infiltration flows and inflate 'Infiltration recall' to near 100%.

7.2 Cleaning

- **Identifier columns dropped** (flow ID, source/destination IP, ports, timestamp): they identify testbed hosts, so a model using them learns 'this IP was the attacker' and cannot transfer.
- **NaN / infinite values** removed (CICFlowMeter produces infinite rates when flow duration is 0).
- **Conflicting duplicates** (identical features, different labels) removed entirely: no model can classify them.
- **Exact duplicates** removed *before* splitting, using a 64-bit hash per row; otherwise copies sit in both train and test.
- **Benign capped** at 600,000 random flows (all attacks kept) to fit 8 GB RAM.
- **Rare classes:** families with fewer than 100 flows (Infiltration 36, Heartbleed 11) are excluded from the multi-class task and reported; they stay in the binary and leave-one-attack-out tests.

Step	Rows
Loaded	2,099,976
NaN / infinite	−5
Conflicting-label duplicates	−2,678

Step	Rows
Exact duplicates	−382,467
After cleaning	1,714,826
After benign cap and rare-class exclusion	882,246
Train / test (random split)	705,796 / 176,450
Train / test (time-block split)	703,770 / 178,476

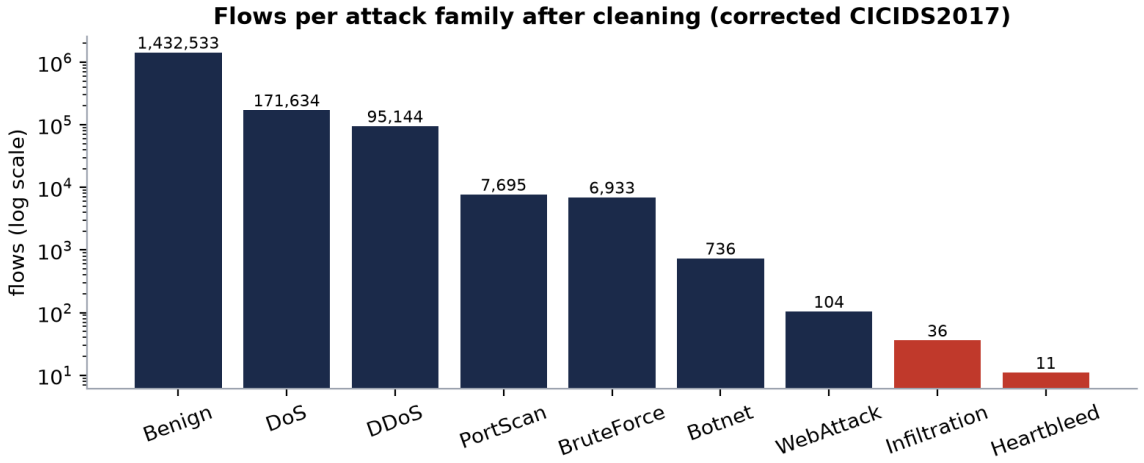


Figure 2. Flows per family after cleaning (log scale). Port scans fall from about 231k to 7.7k flows once ports are dropped, because scan probes are identical SYN→RST flows.

7.3 Train/test splits

Random split: 80/20, stratified by family, seed 42 — what most published work uses.

Time-block split (leakage check): flows are grouped into 5-minute blocks per day, and whole blocks go to either train or test (StratifiedGroupKFold). Neighbouring, near-identical flows can then never straddle the split. Because short attacks span few blocks (DDoS: 5), the splitter tries several shuffles and keeps the split that puts every class with at least two blocks on both sides, with a test fraction close to 20%.

Bug found and fixed: the first time-block run used a single fold, which happened to put all DDoS blocks in train. The test set had 0 DDoS flows, and macro-F1 counted DDoS as a class scoring 0, cutting every model's score by about 1/7. We (1) rewrote the splitter as described above and (2) made macro metrics average only over classes present in the test set, reporting any absent class. The invalid results were archived, not deleted, and the time-block run was repeated.

7.4 Feature selection (training split only)

Fitted once on the training split and reused by every model: (1) remove near-constant features; (2) greedy correlation pruning — walk the features and drop any with |Pearson r| > 0.95 against one already kept (82 → 60 features); (3) Recursive Feature Elimination with a Random Forest (100 trees, depth 20, balanced class weights) on a 200k-row stratified sample, keeping 30.

#	Selected feature (random split)	#	Selected feature
1	Bwd Packet Length Mean	16	Flow Bytes/s
2	Bwd Packet Length Max	17	Fwd RST Flags
3	Fwd Packet Length Mean	18	Total Fwd Packet

#	Selected feature (random split)	#	Selected feature
4	Packet Length Variance	19	Flow IAT Max
5	Total Length of Fwd Packet	20	Fwd IAT Std
6	Fwd Packet Length Max	21	Fwd Seg Size Min
7	Fwd Packet Length Std	22	Bwd IAT Min
8	Fwd PSH Flags	23	Total TCP Flow Time
9	Bwd PSH Flags	24	Bwd IAT Std
10	Bwd Init Win Bytes	25	Fwd Packets/s
11	Down/Up Ratio	26	Flow Duration
12	Fwd Act Data Pkts	27	Fwd IAT Mean
13	Bwd Header Length	28	SYN Flag Count
14	FWD Init Win Bytes	29	FIN Flag Count
15	Fwd Header Length	30	Flow IAT Mean

Examples of pruned features: Total Bwd packets (r = 0.999 with Total Fwd Packet); Total Length of Bwd Packet (r = 0.997 with Total Fwd Packet); Bwd Packet Length Std (r = 0.986 with Bwd Packet Length Max); Fwd IAT Total (r = 0.999 with Flow Duration); Fwd IAT Max (r = 0.999 with Flow IAT Max); Fwd IAT Min (r = 0.965 with Fwd IAT Mean); Bwd IAT Total (r = 0.979 with Flow Duration); Packet Length Max (r = 0.993 with Bwd Packet Length Max).

7.5 Classifiers and hyper-parameters

Each classifier is wrapped in an imbalanced-learn pipeline. Distance- and gradient-based models (LR, SVM, MLP) receive signed-log then standardised features, because flow features span about ten orders of magnitude; tree models use raw features (invariant to monotone transforms, and readable in SHAP plots). Hyper-parameters were tuned once per model with RandomizedSearchCV (12 draws, 3-fold, macro-F1, on a 150k-row stratified sample, strategy 'none') and reused for all imbalance strategies so the strategy comparison is not confounded by tuning.

Model	Search space	Chosen (random split)	Tuning CV F1
Logistic Regression	C from 0.001 to 100 (log-uniform)	C = 70.7	0.968
RBF-SVM	C [0.1, 1000], γ [10^-4, 1] (log-uniform); trained on ≤ 60k rows	C = 25.4, gamma = 0.068	0.990
MLP	layers {64, 128-64, 256-128, 128-64-32}, lr 10^-4–10^-2, α 10^-6–10^-2, {relu, tanh}	activation = relu, alpha = 0.000295, hidden_layer_sizes = [128, 64], learning_rate_init = 0.00112	0.985
Random Forest	trees 100–400, depth {none, 20, 30, 40}, min split 2–10, max features {sqrt, 0.3, 0.5}	max_depth = 40, max_features = sqrt, min_samples_split = 2, n_estimators = 121	0.990
XGBoost	learning rate 0.03–0.3, depth 4–10, trees 150–500, subsample 0.7–1, colsample 0.6–1	colsample_bytree = 0.683, learning_rate = 0.111, max_depth = 8, n_estimators = 340, subsample = 0.953	0.992

7.6 Class-imbalance strategies

- **None:** learn from the natural class mix.
- **Class weights:** loss weighted by inverse class frequency (class_weight='balanced'; balanced sample weights for XGBoost; not available for scikit-learn's MLP, hence 14 rather than 15 runs per split).
- **SMOTE (SafeSMOTE):** runs inside every CV fold and only on training data; lifts each class below 20,000 rows to 20,000 (not full balance, which would create millions of synthetic rows); neighbours are found in signed-log, standardised space and synthetic rows mapped back to raw units; k shrinks automatically for

tiny classes.

7.7 Training protocol

- Per run: 5-fold stratified (or time-grouped) CV on up to 300k training rows to obtain per-fold scores for the significance tests, then a final fit on the full training split and one evaluation on the held-out test split.
- RBF-SVM is trained on a 60k-row proportional sample with a per-class floor (its cost grows roughly with n^2); the MLP on 400k.
- Models, predictions, per-class tables and confusion matrices are saved for every run.

7.8 Evaluation metrics

Metric	Definition	Why it matters
Macro-F1	mean of per-class F1 (classes present in test)	every attack family counts equally
Per-class recall	TP / (TP + FN) per family	exposes failures hidden by accuracy
Benign FPR	benign flows flagged as attack / all benign	alert fatigue
ROC-AUC, PR-AUC	on attack score = 1 − P(benign)	threshold-free separability; PR-AUC suits imbalance
Recall at fixed FPR	best detection rate with FPR ≤ 0.1% or 1%	how an IDS is actually tuned

7.9 Significance tests

A plain paired t-test over CV folds overstates significance because the folds share training data. We used the **Nadeau–Bengio corrected resampled t-test**: $t = \text{mean}(d) / \text{sqrt}((1/k + n_{test}/n_{train}) \cdot \text{var}(d))$, with d the per-fold macro-F1 differences and k = 5, and **McNemar's test** on the two models' errors over the shared test set (exact binomial when fewer than 25 disagreements).

7.10 Explainability, deployability and generalisation tests

- **SHAP:** TreeExplainer on the best tree model; mean |SHAP| per feature and class on 5,000 stratified test flows, a beeswarm plot for the attack direction, and waterfall plots for individual flagged flows.
- **Deployability:** compressed model size; single-flow latency measured one flow at a time with one thread (median, p95, p99 over 1,000 flows); batch throughput on 10,000 flows; ONNX Runtime latency for Random Forest and XGBoost.
- **Leave-one-attack-out:** for each family, a binary XGBoost is trained without that family and tested on all of its flows; also reported: detection at the threshold that gives 1% FPR, and ROC-AUC of unseen-attack vs benign scores.
- **Cross-dataset:** every saved model is applied, unchanged, to corrected CSE-CIC-IDS2018 and scored as benign vs attack.

8. Results

8.1 Random split — all runs

Model	Strategy	Macro-F1	CV macro-F1 (mean ± sd)	Attack recall	Benign FPR	PR-AUC	Train (s)
Random Forest	SMOTE	0.999	0.9905 ± 0.0053	0.9998	0.005%	1.0000	58
XGBoost	SMOTE	0.996	0.9915 ± 0.0078	0.9999	0.008%	1.0000	36
Random Forest	class weights	0.996	0.9902 ± 0.0080	0.9998	0.002%	1.0000	60
XGBoost	none	0.996	0.9902 ± 0.0072	0.9999	0.005%	1.0000	30
Random Forest	none	0.995	0.9900 ± 0.0084	0.9997	0.004%	1.0000	54
MLP	none	0.994	0.9838 ± 0.0090	0.9993	0.018%	1.0000	15
XGBoost	class weights	0.993	0.9923 ± 0.0067	0.9999	0.012%	1.0000	32
RBF-SVM	none	0.991	0.9856 ± 0.0092	0.9994	0.025%	0.9996	1
Logistic Regression	none	0.981	0.9705 ± 0.0076	0.9964	0.116%	0.9994	14
MLP	SMOTE	0.972	0.9761 ± 0.0103	0.9995	0.028%	1.0000	19
RBF-SVM	SMOTE	0.935	0.9842 ± 0.0049	0.9994	0.092%	0.9988	3
RBF-SVM	class weights	0.918	0.9778 ± 0.0104	0.9995	0.143%	0.9981	1
Logistic Regression	SMOTE	0.867	0.8743 ± 0.0033	0.9971	0.388%	0.9991	28
Logistic Regression	class weights	0.789	0.8059 ± 0.0057	0.9986	2.277%	0.9917	30

Test set: 176,450 flows. Sorted by macro-F1.

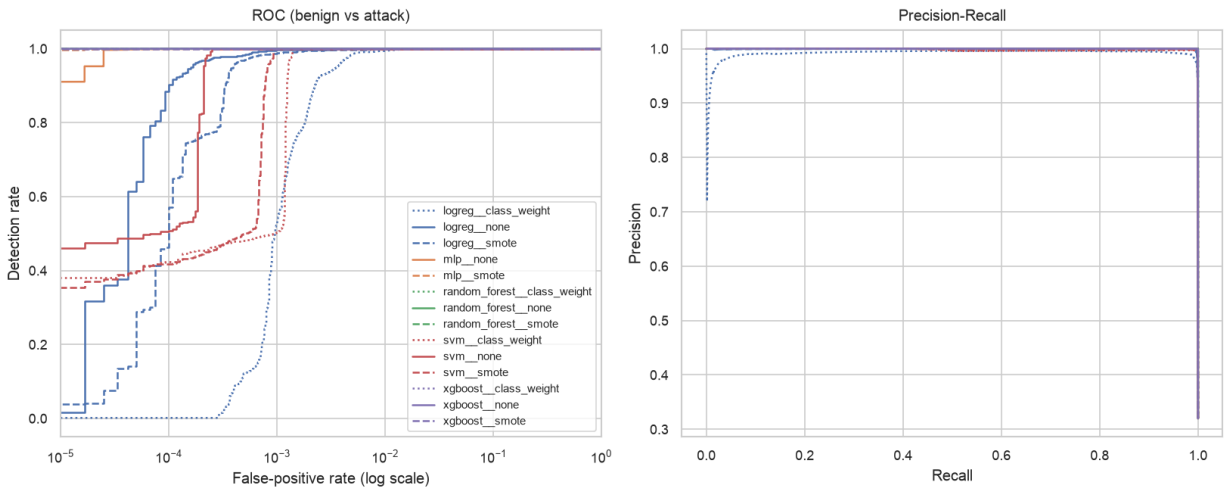


Figure 3. ROC (log-scale FPR) and precision-recall curves for benign vs attack, random split.

8.2 Time-block split — all runs

Model	Strategy	Macro-F1	CV macro-F1 (mean ± sd)	Attack recall	Benign FPR	PR-AUC	Train (s)
Random Forest	SMOTE	0.938	0.9896 ± 0.0113	0.9967	0.005%	1.0000	105
Random Forest	none	0.938	0.9870 ± 0.0083	0.9966	0.005%	1.0000	106
Random Forest	class weights	0.937	0.9904 ± 0.0074	0.9964	0.004%	1.0000	101
XGBoost	SMOTE	0.936	0.9903 ± 0.0064	0.9990	0.006%	1.0000	40

Model	Strategy	Macro-F1	CV macro-F1 (mean ± sd)	Attack recall	Benign FPR	PR-AUC	Train (s)
XGBoost	none	0.935	0.9863 ± 0.0125	0.9990	0.007%	1.0000	34
XGBoost	class weights	0.934	0.9927 ± 0.0077	0.9994	0.007%	1.0000	36
MLP	none	0.928	0.9606 ± 0.0613	0.9966	0.026%	0.9996	11
MLP	SMOTE	0.919	0.9570 ± 0.0595	0.9967	0.019%	0.9994	22
RBF-SVM	none	0.906	0.9908 ± 0.0054	0.9966	0.045%	0.9981	1
Logistic Regression	none	0.890	0.8462 ± 0.1009	0.9986	0.109%	0.9997	12
RBF-SVM	SMOTE	0.871	0.9574 ± 0.0685	0.9969	0.070%	0.9989	2
RBF-SVM	class weights	0.871	0.9897 ± 0.0081	0.9967	0.090%	0.9988	1
Logistic Regression	SMOTE	0.834	0.7905 ± 0.1019	0.9980	0.241%	0.9995	21
Logistic Regression	class weights	0.721	0.6969 ± 0.1176	0.9978	1.617%	0.9969	20

Test set: 178,476 flows; all seven classes present.

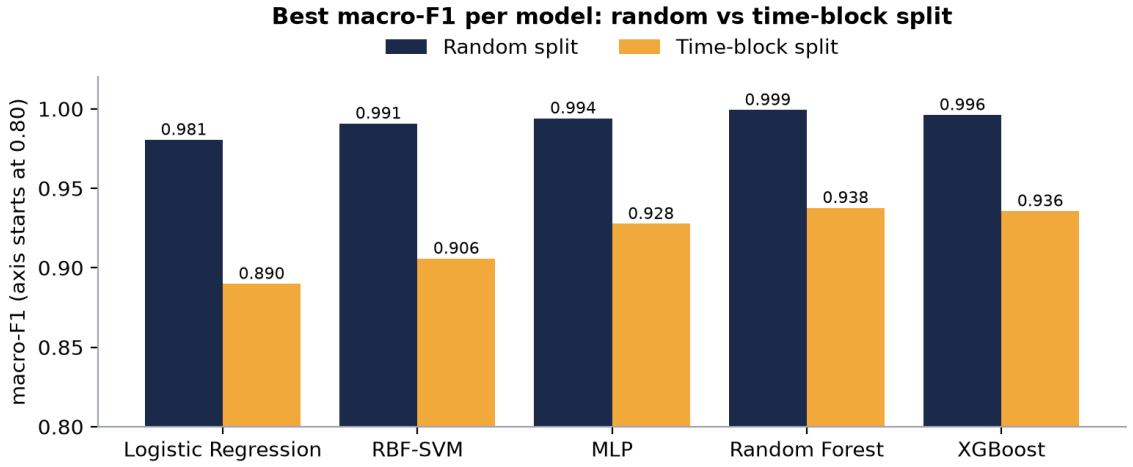


Figure 4. Best macro-F1 per model on the two splits.

Benign-vs-attack detection barely changes under the time-block split (attack recall ≥ 0.996, FPR ≤ 0.05% for the tree models), so leakage between neighbouring flows does not inflate binary detection much on the corrected, de-duplicated data. The multi-class drop from about 0.99 to 0.94 comes almost entirely from one class, shown next.

8.3 Per-class recall

Model (best strategy)	Benign	Botnet	BruteForce	DDoS	DoS	PortScan	WebAttack
Logistic Regression (none)	0.999	1.000	0.995	0.999	0.997	0.741	0.455
RBF-SVM (none)	1.000	1.000	0.997	1.000	0.982	0.966	0.455
MLP (none)	1.000	1.000	0.996	1.000	0.982	0.966	0.455
Random Forest (SMOTE)	1.000	1.000	0.998	1.000	0.982	0.983	0.455
XGBoost (SMOTE)	1.000	1.000	0.998	1.000	0.997	0.983	0.455
Test flows	121,112	162	1,982	46,293	8,847	58	22

Time-block split. Cell colours: ≥ 0.99 blue, 0.90–0.99 light amber, 0.70–0.90 amber, < 0.70 red.

An unseen variant is always missed. The time-block split placed 12 SQL-injection flows in test and only 1 in train. Every model missed all of them (classified as benign or DoS), while web brute force — well represented in training — was caught 10/10. This is the same failure mode as the leave-one-attack-out experiment, on a small scale.

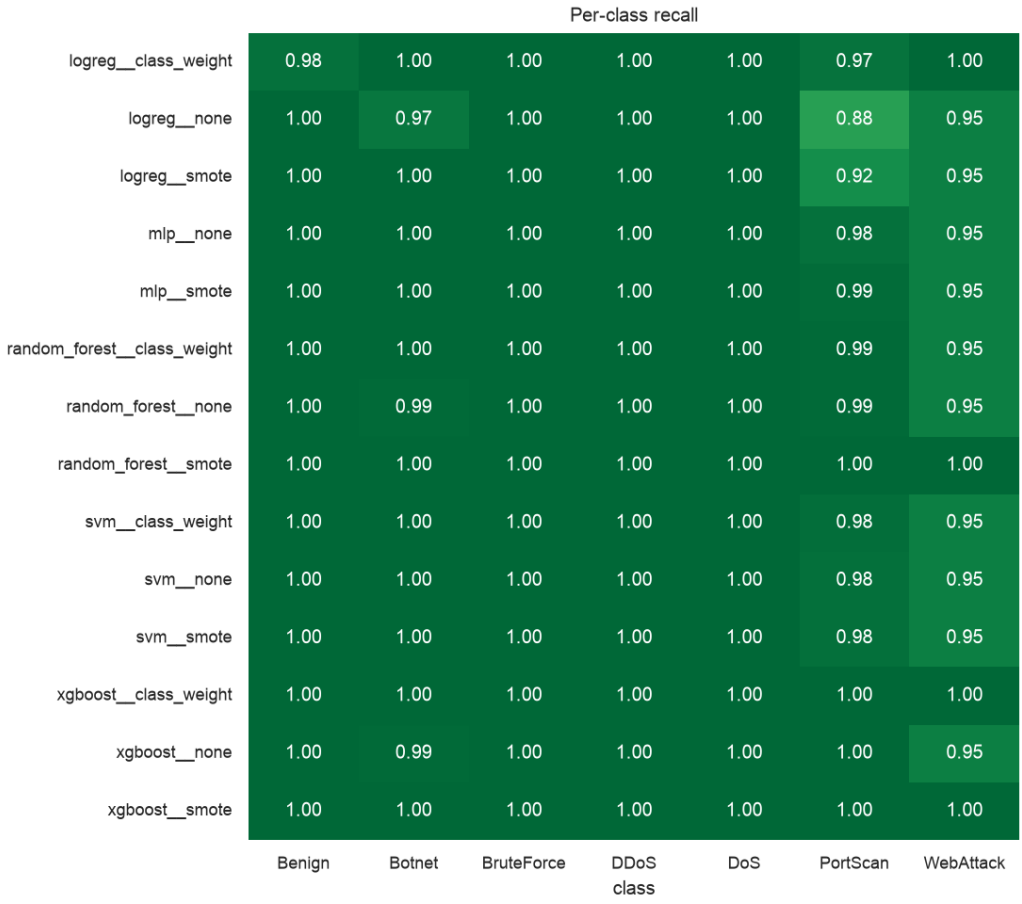


Figure 5. Per-class recall for every run, random split. Rare-class recall is already high without SMOTE.

8.4 Effect of the imbalance strategies

Model	none	class weights	SMOTE	Benign FPR (none → class weights)
Logistic Regression	0.981	0.789	0.867	0.116% → 2.277%
RBF-SVM	0.991	0.918	0.935	0.025% → 0.143%
MLP	0.994	—	0.972	—
Random Forest	0.995	0.996	0.999	0.004% → 0.002%
XGBoost	0.996	0.993	0.996	0.005% → 0.012%

Random split, macro-F1. Class weights push the decision boundary towards the rare classes; for Logistic Regression this multiplies benign false positives about 20-fold. SMOTE gives no statistically significant gain for any model.

8.5 Statistical significance

Pair (strategy 'none')	Δ macro-F1 (test)	Corrected t-test p (random)	Corrected t-test p (time-block)	McNemar p (random)
Random Forest vs XGBoost	-0.0003	0.923	0.955	0.0768

Pair (strategy 'none')	Δ macro-F1 (test)	Corrected t-test p (random)	Corrected t-test p (time-block)	McNemar p (random)
Random Forest vs Logistic Regression	+0.0148	0.005	0.104	2.52e-72
XGBoost vs Logistic Regression	+0.0151	0.011	0.113	6.98e-75
Random Forest vs MLP	+0.0013	0.192	0.564	1.01e-08
XGBoost vs RBF-SVM	+0.0048	0.584	0.529	1.3e-10
MLP vs RBF-SVM	+0.0033	0.856	0.525	1

Random Forest and XGBoost are statistically tied ($p = 0.92$ and 0.96). Ensembles beat Logistic Regression on the random split ($p = 0.005$), but no pairwise difference is significant on the time-block split at $\alpha = 0.05$. McNemar's test flags tiny differences as significant because the test set is large (≈ 176k flows); the corrected t-test is the more conservative and more appropriate test here.

8.6 Deployability

Model	Size	Train (s)	Latency median (ms)	p99 (ms)	Batch (flows/s)	ONNX median (ms)
Logistic Regression	3 KB	14	0.157	0.23	3,533,986	—
RBF-SVM	106 KB	1	0.231	0.32	23,443	—
MLP	148 KB	15	0.183	0.55	1,525,233	—
Random Forest	2.7 MB	54	2.782	12.09	366,880	0.011
XGBoost	857 KB	30	0.304	9.33	273,526	0.022

Random split, strategy 'none'; single-flow latency measured single-threaded on the M1. SVM training time is small only because it was capped at 60k rows.

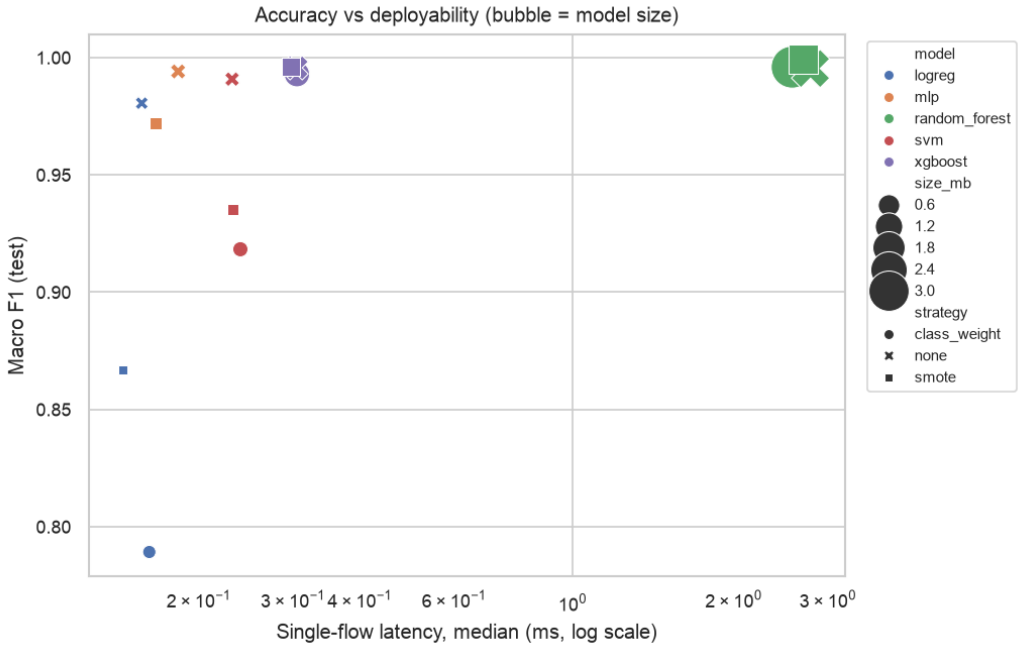


Figure 6. Accuracy versus single-flow latency (bubble size = model size).

XGBoost reaches Random-Forest-level accuracy at 9× lower latency; exported to ONNX Runtime, both tree models classify a flow in about 0.01–0.02 ms, fast enough for inline use. The RBF-SVM is quick per flow but the slowest in batch, because every prediction touches many support vectors. These numbers are classification cost only: a flow can be classified only once CICFlowMeter closes it (FIN/RST or a 120-second timeout), which dominates real detection delay.

8.7 Explainability (SHAP)

Rank	Feature	Mean |SHAP|
1	Bwd Packet Length Mean	0.741
2	Bwd Init Win Bytes	0.721
3	Fwd Packet Length Max	0.657
4	Packet Length Variance	0.626
5	Bwd Packet Length Max	0.547
6	Fwd Packet Length Std	0.443
7	Fwd Packet Length Mean	0.429

Rank	Feature	Mean |SHAP|
8	Total Length of Fwd Packet	0.327
9	Down/Up Ratio	0.279
10	Bwd PSH Flags	0.240

XGBoost (SMOTE), random split, averaged over all classes.

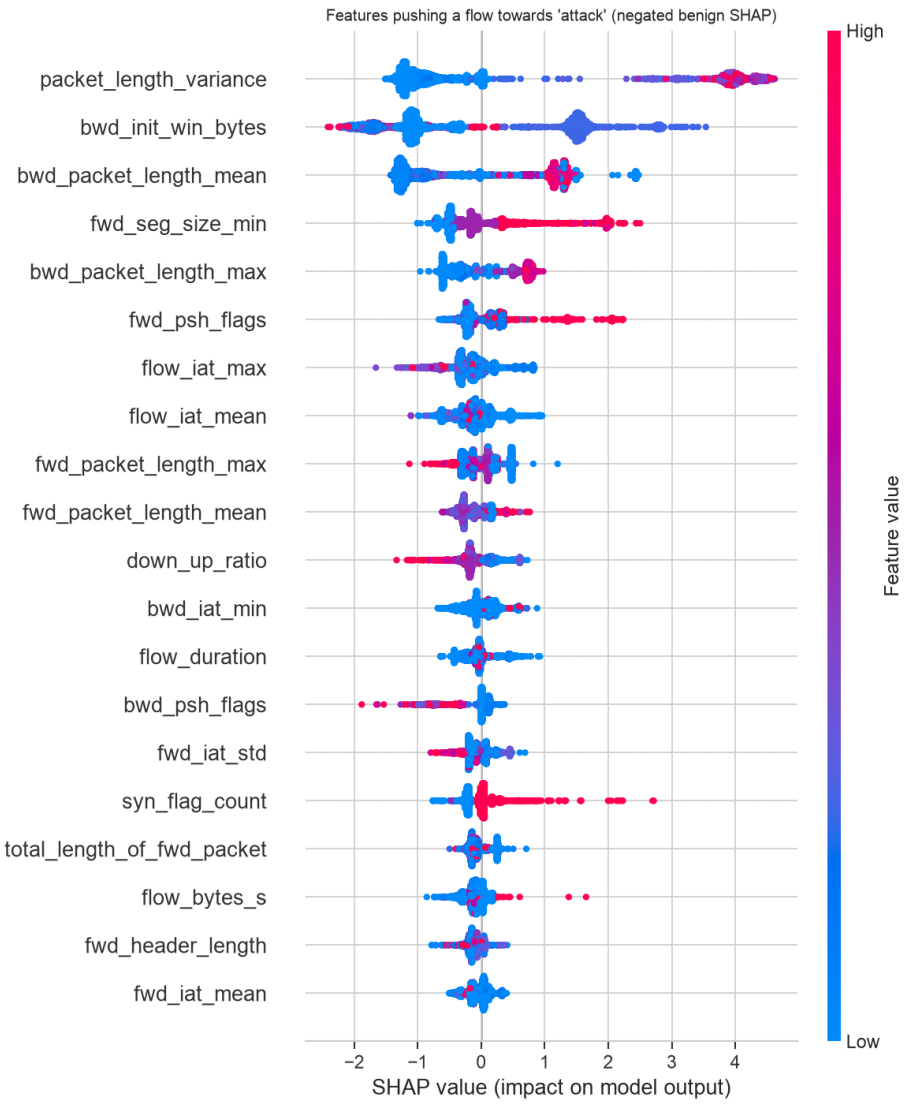


Figure 7. SHAP beeswarm: features pushing a flow towards 'attack'. Red = high feature value.

Packet-length statistics dominate, which is defensible: floods, scans and web attacks have characteristic request and response sizes. But **Bwd Init Win Bytes ranks #2**. The initial TCP window size reflects the operating system and configuration of the hosts in the testbed, not the attack; a model that leans on it partly learns the lab. This is consistent with the cross-dataset collapse in §8.9.

8.8 Leave-one-attack-out (unseen attack families)

Held-out family	Flows	Detected (0.5)	Detected (1% FPR)	ROC-AUC vs benign	Detected (0.5), time-block
Botnet	736	0.0%	82.6%	0.987	0.0%
BruteForce	6,933	0.0%	12.9%	0.967	0.0%
DDoS	95,144	98.8%	100.0%	1.000	99.9%

Held-out family	Flows	Detected (0.5)	Detected (1% FPR)	ROC-AUC vs benign	Detected (0.5), time-block
DoS	171,634	0.2%	99.2%	0.999	0.5%
Heartbleed	11	0.0%	90.9%	0.997	9.1%
Infiltration	36	11.1%	83.3%	0.980	0.0%
PortScan	7,695	1.7%	14.8%	0.956	1.1%
WebAttack	104	1.0%	14.4%	0.970	1.0%

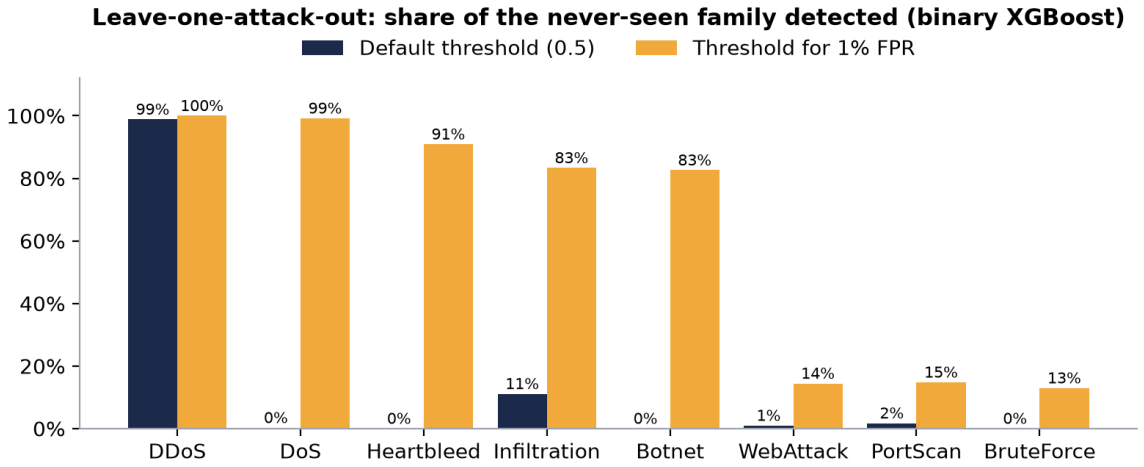


Figure 8. Share of each never-seen family detected by a binary XGBoost (random split).

At the default threshold the detector almost never flags a family it has not seen — the exception is DDoS, which resembles DoS. The attack scores still rank unseen flows above benign (ROC-AUC mostly 0.92–1.00), so re-tuning the threshold recovers DoS, Botnet, Heartbleed and Infiltration; port scans, brute force and web attacks stay below about 15% even at 1% FPR. The 'detects zero-day attacks' claim does not hold as deployed.

8.9 Cross-dataset generalisation (CSE-CIC-IDS2018, no retraining)

Model	In-dist. binary F1	2018 F1	2018 recall	2018 FPR	2018 ROC-AUC	Recall @ 1% FPR
Logistic Regression	0.9970	0.418	26.8%	3.79%	0.829	17.0%
RBF-SVM	0.9994	0.059	3.1%	5.70%	0.378	0.6%
MLP	0.9994	0.018	0.9%	0.09%	0.812	5.8%
Random Forest	0.9998	0.006	0.3%	0.09%	0.651	22.1%
XGBoost	0.9999	0.005	0.3%	0.08%	0.580	3.5%

Random-split models, strategy 'none'. Results for the other strategies and for the time-block models are in results/.../cross_dataset__cicids2018_fixed.csv and show the same pattern.

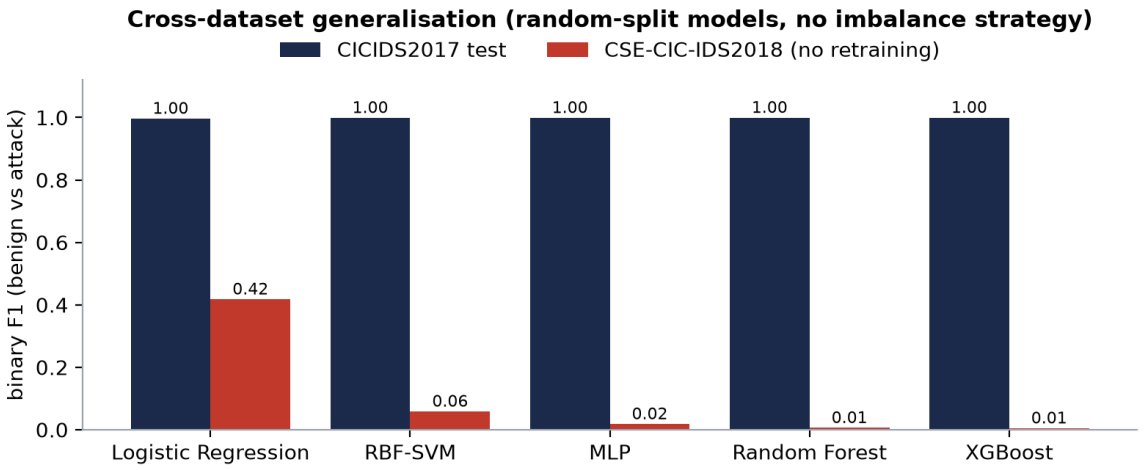


Figure 9. Binary F1 in-distribution vs on CSE-CIC-IDS2018.

Model	Benign flagged	Botnet flagged	BruteForce flagged	DDoS flagged	DoS flagged
Logistic Regression	3.8%	99.3%	0.0%	62.3%	1.5%
RBF-SVM	5.7%	5.4%	0.0%	6.3%	1.2%
MLP	0.1%	4.6%	0.0%	0.0%	1.2%
Random Forest	0.1%	0.6%	0.0%	0.0%	0.5%
XGBoost	0.1%	0.0%	0.0%	0.0%	0.4%

Share of each 2018 family flagged as attack ('Benign flagged' is the false-positive rate).

The two strongest in-distribution models detect only 0.3% of 2018 attacks. Logistic Regression transfers best (F1 0.42, ROC-AUC 0.83), and it is the only model that still catches most Botnet and DDoS flows. Even with the threshold re-tuned for 1% FPR, no model exceeds 22% recall, so this is genuine distribution shift rather than a threshold problem. The more flexible the model, the more it fits capture-specific details of CICIDS2017.

9. Hypotheses: Verdicts

#	Hypothesis	Verdict	Evidence
H1	Ensembles beat LR and match or exceed SVM / MLP	Partly supported	RF / XGBoost rank first; beat LR on the random split ($p = 0.005$); gaps to MLP / SVM not significant
H2	Rare-class recall low before SMOTE	Not supported	Rare-class recall already high without SMOTE; SMOTE gives no significant gain
H3	RF / XGBoost best accuracy-latency; SVM highest training cost	Partly supported	XGBoost: RF-level F1 at 9× lower latency; SVM training cost untested (capped at 60k rows), but it has the lowest batch throughput
H4	Accuracy drops on an independent dataset	Strongly supported	RF / XGBoost: binary F1 from 0.9998 to ≤ 0.01; 0.3% of 2018 attacks detected

Answers to the research questions

- **RQ1:** Random Forest and XGBoost give the best accuracy / FPR trade-off (macro-F1 ≈ 0.99–1.00, benign FPR ≤ 0.01%), statistically tied with each other.
- **RQ2:** XGBoost is the most suitable for near-real-time use; with ONNX Runtime, tree models classify a flow in about 0.01–0.02 ms.
- **RQ3:** Packet-length statistics dominate and are defensible; the high rank of Bwd Init Win Bytes is a testbed artefact.
- **RQ4:** Performance collapses on a different network: the benchmark measures fit to the capture, not transferable attack detection.

10. Discussion and Limitations

- **Why are in-distribution scores so high?** After correcting labels and removing duplicates, most CICIDS2017 attacks are produced by a handful of tools against a few hosts, so their flows are extremely regular. High accuracy here says little about real-world performance — which is exactly what the generalisation tests show.
- **Single seed:** every result comes from seed 42; CV standard deviations (§8.1–8.2) indicate the variance.
- **Small classes:** WebAttack (≈ 104 flows), Heartbleed (11) and Infiltration (36) results are indicative only.
- **2018 subset:** 5 of 10 days, benign sampled at 5%; target precision is optimistic, recall and FPR are not affected.
- **SVM capped** at 60k training rows, so its training-cost hypothesis is not fully tested.
- **Latency is classification cost only**, on a laptop; end-to-end delay is dominated by flow aggregation.
- **Tuning budget:** 12 random draws per model; a larger search could change rankings slightly but not the generalisation findings.

11. Problems Encountered and How We Solved Them

Problem	Cause	Fix
Proposal's cross-dataset plan unworkable	NSL-KDD features cannot be aligned; NetFlow-v2 has no CICIDS2017	Corrected CSE-CIC-IDS2018 with identical features
2018 archive too large (9.7 GB) for the disk	~10 GB free	Stream single days via HTTP range requests; keep attacks + 5% benign

Problem	Cause	Fix
XGBoost failed to import	OpenMP runtime missing on macOS	brew install libomp
Random Forest latency 13.6 ms per flow	n_jobs=-1 starts a thread pool on every one-row call	Measure single-flow latency single-threaded
72k flows labelled 'Infiltration - Portscan'	Port scan run from the compromised host	Mapped to PortScan (behaviour-based families)
231k port-scan flows → 7.7k	Scan probes are identical once ports are dropped	Expected; reported as a finding
Time-block test set had 0 DDoS flows	Single group fold; DDoS spans only 5 blocks	Splitter searches shuffles for full class coverage; macro metrics over present classes; reran
All 2018 downloads died at once	Laptop slept after the first run; sockets dropped	Retries with backoff and timeouts; run under caffeinate
The full run aborted on a missing dataset	One failing step stopped the chain	Steps log errors and continue
Leave-one-attack-out reused an old cache	Binary time-block cache built by the buggy splitter	Cache deleted and step rerun

12. Conclusions and Future Work

We built a reproducible, leakage-safe benchmark of five classifiers for flow-based intrusion detection on corrected CICIDS2017 and went beyond accuracy to significance, deployability, explainability and generalisation. The answer to 'which model should a network deploy?' has two parts:

- **For attacks like those seen in training:** XGBoost — macro-F1 ≈ 0.996, benign FPR ≈ 0.005%, about 0.02 ms per flow with ONNX.
- **For a new network or new attacks:** none of these models can be trusted out of the box. Near-perfect benchmark scores measure how well a model learns the lab, not how well it protects a network.

Recommendations

- Retrain or adapt the detector on traffic from the network where it will be deployed.
- Remove host- and testbed-specific features (such as initial TCP window sizes) and re-test generalisation.
- Choose alert thresholds on validation data for a target false-positive rate instead of using 0.5.
- Report per-class, time-separated and cross-dataset results, not only random-split accuracy.

Future work

- Domain adaptation or training on several datasets at once; federated training across organisations.
- Feature sets that exclude host-specific fields; repeat the study with multiple seeds.
- A live demo that replays flows through the saved model and shows alerts with SHAP reasons; SDN integration.

Appendix A. Reproducing the Results

```
brew install python@3.12 libomp
python3.12 -m venv .venv && source .venv/bin/activate
pip install -e ".[notebooks,onnx,dev]"

# smoke test on synthetic data (~2 min)
python -m ids.synthetic && ids all --config configs/smoke.yaml && pytest

# data
python scripts/fetch_data.py cicids2017
caffeinate -i python scripts/fetch_data.py cicids2018

# full experiments (~1 h per split on an M1)
ids all                      # random split
ids all --split time_block   # time-block split

# documents
python scripts/build_assessment_deck.py
python scripts/build_report.py
```

Pipeline steps

Command	What it does	Main outputs
ids prepare	load, clean, split, select features	data/processed/…/train|test.parquet, meta.json
ids benchmark	tune, 5-fold CV, final fit, test evaluation for every run	runs/*.json, *_per_class.csv, *_preds.parquet, models/
ids stats	corrected t-test + McNemar for every model pair	significance.csv
ids deploy	size, latency, throughput, ONNX	deployability.csv, latency_vs_f1.png
ids explain	SHAP on the best tree model	shap/
ids cross	evaluate all models on CSE-CIC-IDS2018	cross_dataset__cicids2018_fixed*.csv
ids loao	leave-one-attack-out	leave_one_attack_out.csv
ids report	summary tables and figures	summary.csv/.md, roc_pr.png, per_class_*.png

Appendix B. Configuration (configs/default.yaml)

```
# Single source of truth for every experiment. Change values here, not in notebooks,
# so that every model sees exactly the same data and settings (Gap 1 in the proposal).

seed: 42

paths:
  raw: data/raw
  processed: data/processed
  results: results

dataset:
  name: cicids2017_fixed          # corrected CICIDS2017 (Engelen et al. / Liu et al. CNS
2022)
  dir: data/raw/cicids2017_fixed  # folder holding monday.csv ... friday.csv
  # "Attempted" flows contain no real attack payload; the dataset authors say to treat them
as benign.
  attempted_as_benign: true

preprocess:
  drop_duplicates: true
  near_zero_variance: 1.0e-8
```

```
  # Benign flows vastly outnumber attacks. Keeping all of them is not needed to learn
  # the benign class and does not fit in 8 GB RAM once SMOTE runs, so cap them.
  max_benign_rows: 600000
  # Multi-class only: families smaller than this cannot be measured reliably (Heartbleed has
  # ~11 flows, i.e. 2 in a 20% test split). They stay in the binary and LOAO experiments.
  min_class_rows: 100

split:
  test_size: 0.2
  # random     -> stratified random split over flows (what most papers do)
  # time_block -> flows grouped into time buckets so near-identical neighbouring flows
  #               cannot land on both sides of the split (leakage check)
  strategy: random
  time_block_minutes: 5
  cv_folds: 5
  # per-fold CV for the significance tests runs on at most this many training rows
  cv_sample_rows: 300000

features:
  corr_threshold: 0.95
  # Number of features kept after correlation pruning, ranked by RF importance.
  # null keeps everything that survives correlation pruning.
  top_k: 30
  rank_sample_rows: 200000

imbalance:
  # strategies compared in the benchmark: none | class_weight | smote
  strategies: [none, class_weight, smote]
  # SMOTE raises each minority class to at least this many rows (not full balance: that
  # would turn ~600k benign rows into millions of synthetic rows).
  smote_min_rows: 20000

task: multiclass   # multiclass | binary

models:
  # rows used to fit each model; SVM (RBF) is quadratic in n so it gets a hard cap
  max_train_rows:
    logreg: null
    random_forest: null
    xgboost: null
    svm: 60000
    mlp: 400000
  search:
    enabled: true
    n_iter: 12            # RandomizedSearchCV budget per model (fixed and documented)
    cv_folds: 3
    sample_rows: 150000   # tuning runs on a stratified subsample to stay within the time
budget
  list: [logreg, random_forest, xgboost, svm, mlp]

evaluation:
  fpr_targets: [0.001, 0.01]   # report detection rate at these false-positive rates

deploy:
  latency_samples: 1000
  batch_size: 10000

explain:
  model: xgboost
  background_rows: 2000
  explain_rows: 5000
  local_examples: 5
```

```
cross_dataset:
  # target dataset evaluated WITHOUT retraining (binary benign/attack only, since
  # attack taxonomies differ between datasets)
  target: cicids2018_fixed
  target_dir: data/raw/cicids2018_fixed

loao:
  # leave-one-attack-out: binary detector trained without one family, tested on it
  model: xgboost
  strategy: none
  max_train_rows: 400000
```

Appendix C. References

- Sharafaldin, Lashkari, Ghorbani (2018). Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. ICISSP.
- Engelen, Rimmer, Joosen (2021). Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study. IEEE S&P Workshops.
- Liu, Engelen, Lynar, Essam, Joosen (2022). Error Prevalence in NIDS Datasets: A Case Study on CIC-IDS-2017 and CSE-CIC-IDS-2018. IEEE CNS.
- Sarhan, Layeghy, Portmann (2021). Towards a Standard Feature Set for Network Intrusion Detection System Datasets.
- Nadeau, Bengio (2003). Inference for the Generalization Error. Machine Learning 52.
- Dietterich (1998). Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms. Neural Computation.
- Chawla, Bowyer, Hall, Kegelmeyer (2002). SMOTE: Synthetic Minority Over-sampling Technique. JAIR 16.
- Breiman (2001). Random Forests. Machine Learning 45.
- Chen, Guestrin (2016). XGBoost: A Scalable Tree Boosting System. KDD.
- Lundberg, Lee (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS.